B2 Code design

Pure functions, dataclasses, when to use classes, configuration objects.

B2-L1 Pure functions, side effects at the edges

The pure core is tested with three rows and a fixed year. The edge is a few lines that are easy to read.

```python
def weighted_mean(values: list[float], weights: list[float]) -> float:
    return sum(v * w for v, w in zip(values, weights, strict=True)) / sum(weights)
```

```
read (impure) -> transform, compute (pure) -> write (impure)
```

What makes a function pure?	Its result depends only on its arguments, and it changes nothing outside itself.
Three hidden side effects in data code	Mutating arguments, reading globals, reading the clock or randomness inside the function.
Return a new DataFrame with an extra column	df.assign(col=...). df["col"] = ... changes the caller's frame.
Program shape	Read at the edge, compute in a pure core, write at the edge.

B2-L2 Dataclasses as configuration objects

A frozen dataclass names every setting, rejects unknown ones, and checks values once, when you build it.

```python
from dataclasses import dataclass, field


@dataclass(frozen=True)
class ReportConfig:
    weight_col: str
    min_base: int = 50
    waves: tuple[str, ...] = ()

    def __post_init__(self) -> None:
        if self.min_base <= 0:
            raise ValueError(f"min_base must be positive, got {self.min_base}")
```

Why a frozen dataclass for settings?	Named, typed fields, unknown keys rejected, and no assignment after construction.
Change one setting on a frozen config	dataclasses.replace(cfg, min_base=30) returns a new object.
Are dataclass type hints checked at runtime?	No. Validate or convert in `__post_init__`, or use pydantic for external input.
Empty list default on a dataclass field	field(default_factory=list). A plain [] raises ValueError at class definition.

B2-L3 When a class earns its place

Functions by default. A class when state lives across calls, an invariant needs guarding, or several methods share the same data.

```python
class Normaliser:
    def __init__(self, col):
        self.col = col

    def run(self, df):
        return df.assign(**{self.col: df[self.col] / df[self.col].sum()})
```

A class with `__init__` and one run method	A function in disguise. Replace it with a function.
Four reasons a class earns its place	State across calls, an invariant to protect, several operations on the same data, a shared interface.
The deletion test	Turn the methods into functions. If nothing breaks, keep the functions.
Settings with no behaviour	A frozen dataclass.